

实验报告

数字逻辑与计算机组成实验

lab02: 简易 ALU 运算器

姓名: 法童

学号: 251250046

邮箱: 251250046@smail.nju.edu.cn

实验时间: 2026-09-06

1 实验目的

1. 掌握用行为级 Verilog 描述 4 位简易 ALU，通过 3 位功能选择端 ALUctr (casez) 完成加、减、按位取反、与、或、异或、有符号小于比较与相等判断共 8 种运算。
2. 理解并正确产生 cf (进位/借位)、zero (结果全零)、of (有符号溢出) 三个标志，能区分无符号进位与有符号溢出两种不同含义。
3. 学会编写自校验 (self-checking) testbench 做覆盖测试，并把 ALU 封装为 top 模块映射到开发板的拨码开关、LED 与七段数码管完成上板验证。

2 实验原理

本实验的 ALU 是一个纯组合逻辑模块：读入两个 4 位操作数 A 、 B 与 3 位功能选择 ALUctr，按功能表产生 4 位结果 F ，并输出进位/借位 cf、结果为零 zero、有符号溢出 of 三个标志。运算与标志含义见表 1。

表 1: ALU 功能表 (ALUctr 与运算、标志的关系)

ALUctr	运算	标志处理
000	$F = A + B$	cf= 进位, of、zero 由结果产生
001	$F = A - B$	cf= 借位, of、zero 由结果产生
010	$F = \sim A$	清零
011	$F = A \& B$	清零
100	$F = A B$	清零
101	$F = A \oplus B$	清零
110	$F = (A < B)$ 有符号比较	清零
111	$F = (A == B)$	清零

其中加/减运算用 5 位中间量捕获进位或借位：

$$\{\text{cf}, F\} = A + B \quad (\text{加法}) \quad \{\text{cf}, F\} = A - B \quad (\text{减法, cf 为借位})$$

cf 表示无符号意义上的进位/借位；of 表示把 A 、 B 看成二进制补码时的有符号溢出——加法时两操作数同号而结果异号、减法时两操作数异号而结果与 A 异号即溢出。该溢出逻辑由 4 位结果的符号位 $F[3]$ 直接比较得到，无需扩展位宽。逻辑运算 (010–111) 不产生进位，标志统一清零，避免上一运算的标志残留（见 §4 的代码）。

3 实验环境与器材

- 开发板：Nexys A7-100T (FPGA 型号 xc7a100tcsg324-1)
- 开发工具：Vivado 2020.2
- 硬件描述语言：Verilog HDL

4 设计思路

4.1 ALU 核心模块 (alu_s)

模块端口：A[3:0]、B[3:0]、ALUctr[2:0] 输入，F[3:0]、cf、zero、of 输出。原理框图如下：

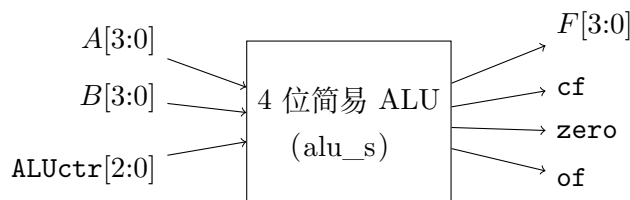


图 1: ALU 模块原理框图

核心逻辑用组合 `always + casez` 描述。为杜绝标志残留，进入 `always` 后先统一清零 `F` 与三个标志，算术分支再按需赋值：

```

1  always @(*) begin
2      F = 4'd0; cf = 1'b0; zero = 1'b0; of = 1'b0; // reset every
      entry
3      casez (ALUctr)
4          3'b000 : begin
5              {cf, F} = A + B; // cf = carry-out
6              of = (A[3]==B[3]) && (F[3]!=A[3]);
7              zero = ~|F;
8          end
9          3'b001 : begin
10             {cf, F} = A - B; // cf = borrow
11             of = (A[3]!=B[3]) && (F[3]!=A[3]);
12             zero = ~|F;
13         end
14         3'b010 : F = ~A;
  
```

```

15      3'b011 : F = A & B;
16      3'b100 : F = A | B;
17      3'b101 : F = A ^ B;
18      3'b110 : F = $signed(A) < $signed(B);          // signed less-
              than
19      3'b111 : F = (A == B);
20  endcase
21 end

```

说明：加/减时 F 按无符号 4 位取结果低 4 位， cf 取第 5 位；两处溢出判定分别对应表 1 的补码规则。“有符号小于”运算用 $\$signed()$ 将 4 位输入按补码解释后再比较，结果为 1 或 0。

4.2 顶层映射 (top)

top 是综合目标,把逻辑端口连到板级引脚: $A \leftarrow SW[10:7]$ 、 $B \leftarrow SW[6:3]$ 、 $ALUctr \leftarrow SW[2:0]$; $F \rightarrow LED[10:7]$, $cf/zero/of \rightarrow LED[0..2]$; F 再送 $bcd7seg$ 译成一位七段数码管显示。系统框图如下:

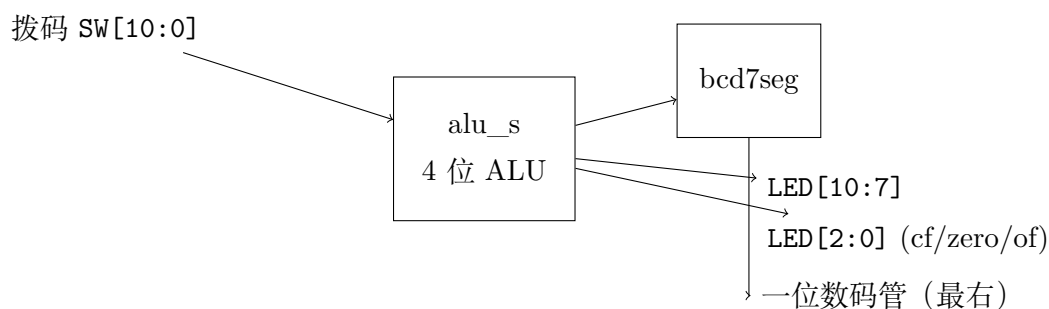


图 2: 系统顶层连接框图

一个实现细节是结果 F 为带符号补码,而一位数码管无法直接显示符号。 $top.v$ 里对负数求补码绝对值后再送译码:

```

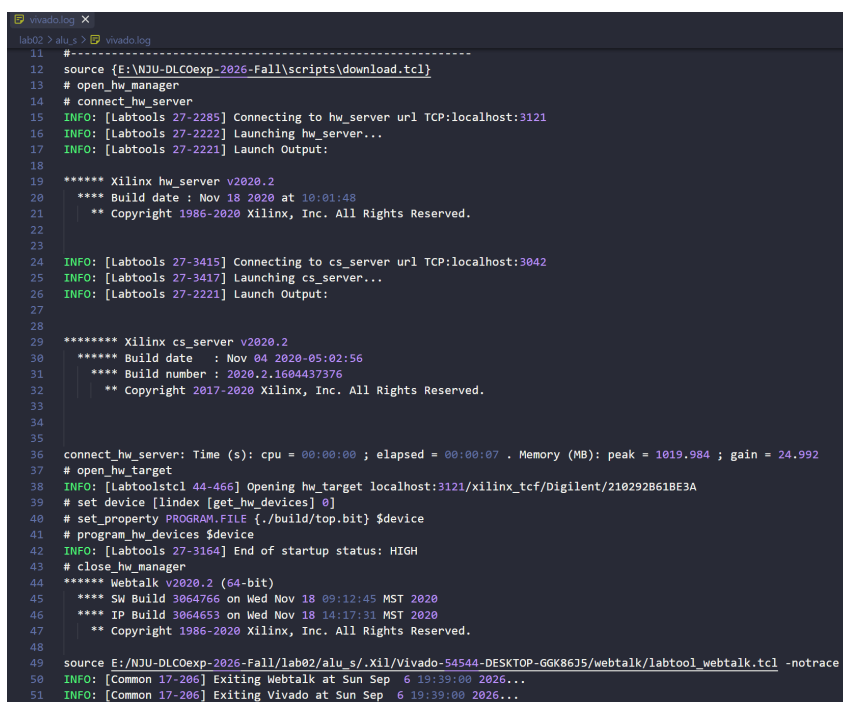
1 bcd7seg bcd7seg1(.b((F^4{F[3]})+F[3]), .h({CG,CF,CE,CD,CC,CB,CA}));
2 assign AN = 8'b11111110;    // enable the rightmost digit only
3 assign DP = 1'b1;          // turn off the decimal point

```

$F[3] = 1$ 时 $(F \oplus 1111) + 1$ 恰为 $|F|$, 故数码管直接显示结果的数值大小; 判读符号以 $LED[3]$ 位 ($F[3]$) 为准。

5 实验过程

1. 编写功能模块 `alu_s.v` (行为级 `casez` 描述 8 种运算与标志) 与七段译码模块 `bcd7seg.v`, 再按 `top` 包装模式编写 `top.v`, 把逻辑端口映射到 SW/LED/七段数码管。
2. 编写约束文件: 从根目录 `nexysa7.xdc` 复制得到 `alu_s.xdc`, 仅取消本次用到引脚的注释 (SW[8]、SW[9] 保持 LVCMOS18), 未使用输出在 `top.v` 中置 0 并补齐约束。
3. 编写自校验 testbench `sim/tb.v`, 运行 `sim.bat` 做功能仿真, 日志输出 [PASS] all 200 vectors OK。
4. 综合与实现: 运行 `build.bat` 生成比特流 `build/top.bit`; 运行 `prog.bat` 下载到开发板, 逐项拨动开关验证。



```

lab02 2 alu_s > vivado.log
11 #-----
12 source {E:\NJU-DLCoexp-2026-Fall\scripts\download.tcl}
13 # open_hw_manager
14 # connect_hw_server
15 INFO: [Labtools 27-2285] Connecting to hw_server url TCP:localhost:3121
16 INFO: [Labtools 27-2222] Launching hw_server...
17 INFO: [Labtools 27-2221] Launch Output:
18
19 ***** Xilinx hw_server v2020.2
20 **** Build date : Nov 18 2020 at 10:01:48
21 ** Copyright 1986-2020 Xilinx, Inc. All Rights Reserved.
22
23
24 INFO: [Labtools 27-3415] Connecting to cs_server url TCP:localhost:3042
25 INFO: [Labtools 27-3417] Launching cs_server...
26 INFO: [Labtools 27-2221] Launch Output:
27
28
29 ***** Xilinx cs_server v2020.2
30 ***** Build date : Nov 04 2020-05:02:56
31 **** Build number : 2020.2.1604437376
32 ** Copyright 2017-2020 Xilinx, Inc. All Rights Reserved.
33
34
35
36 connect_hw_server: Time (s): cpu = 00:00:00 ; elapsed = 00:00:07 . Memory (MB): peak = 1019.984 ; gain = 24.992
37 # open_hw_target
38 INFO: [Labtoolstcl 44-466] Opening hw_target localhost:3121/xilinx_tcf/Digilent/210292861BE3A
39 # set device [lindex [get_hw_devices] 0]
40 # set_property PROGRAM.FILE {./build/top.bit} $device
41 # program_hw_devices $device
42 INFO: [Labtools 27-3164] End of startup status: HIGH
43 # close_hw_manager
44 ***** Webtalk v2020.2 (64-bit)
45 **** SW Build 3064766 on Wed Nov 18 09:12:45 MST 2020
46 **** IP Build 3064653 on Wed Nov 18 14:17:31 MST 2020
47 ** Copyright 1986-2020 Xilinx, Inc. All Rights Reserved.
48
49 source E:/NJU-DLCoexp-2026-Fall/lab02/alu_s/.Xil/Vivado-54544-DESKTOP-GGK86J5/webtalk/labtool_webtalk.tcl -notrace
50 INFO: [Common 17-206] Exiting Webtalk at Sun Sep 6 19:39:00 2026...
51 INFO: [Common 17-206] Exiting Vivado at Sun Sep 6 19:39:00 2026...

```

图 3: Vivado 综合与实现完成

6 测试方法

验证从两方面进行: 一是用 Vivado 做功能仿真观察波形并自动比对, 二是下载到开发板实测; 不依赖课程 OJ 判题。

仿真（自校验）：tb.v 内置一个独立的参考模型，用 5 位运算对每种操作数组合重算期望的 F /标志，再与 DUT 输出逐项比对，不一致即打印 MISMATCH 并累计错误，从而避免“以 DUT 验证 DUT”。测试信号的选择上，操作数取 5 个代表性值——全零（0，触发 zero）、+1（触发进位/借位）、最大正数 +7、最小负数 -8（补码边界）与全 1（-1，借位/比较边界），两两组合共 5×5 组，每组遍历 8 种运算，合计 200 组输入全部自动比对。

上板实测： $A =$ 拨码 SW[10:7]， $B =$ SW[6:3]，运算选择 ALUctr=SW[2:0]。SW[2:0] 依次置 000–111 对应表 1 的 8 种运算，拨入若干组 A/B ，观察 LED[10:7]（ F 二进制）、LED[0]/LED[1]/LED[2]（cf/zero/of）与最右一位数码管显示的数值，重点核对加减法的进位/借位、负数显示与溢出标志。

7 实验结果

7.1 仿真结果

运行 sim.bat，testbench 对 200 组输入全部自动比对通过，Vivado 日志输出：

[PASS] all 200 vectors OK.

波形上逐组验证：ALUctr=000/001 时加法/减法结果与 cf（进位/借位）正确；zero 在结果全零时拉高；对有符号溢出的组合（如 $7+1$ 、 $-8-1$ ）of 正确置 1；010–111 各逻辑/比较运算输出与真值一致。

7.2 上板结果

下载比特流后实测（ALUctr 由 SW[2:0] 给定，SW[2:0]=000/001/011 分别为加/减/与）：

- 加法 $A = 2, B = 3$ ： $F = 5$ ，LED[10:7]=0101，cf/zero/of 均熄灭，数码管显示 5；
- 加法 $A = 7, B = 1$ ： $F = 1000$ （已超出补码范围 +7，of 应置 1），LED[2] 点亮，数码管显示 8；
- 减法 $A = 3, B = 5$ ： $F = 1110$ （-2），LED[0]（借位）点亮，数码管显示负数绝对值 2；
- 与运算 $A = 7, B = 5$ ： $F = 0101$ ，LED[10:7]=0101，三个标志熄灭，数码管显示 5。

现象与仿真及表 1 真值一致（若 $|F| > 9$ ，数码管超出 0–9 译码范围会熄灭，此时以 LED 二进制为准）。

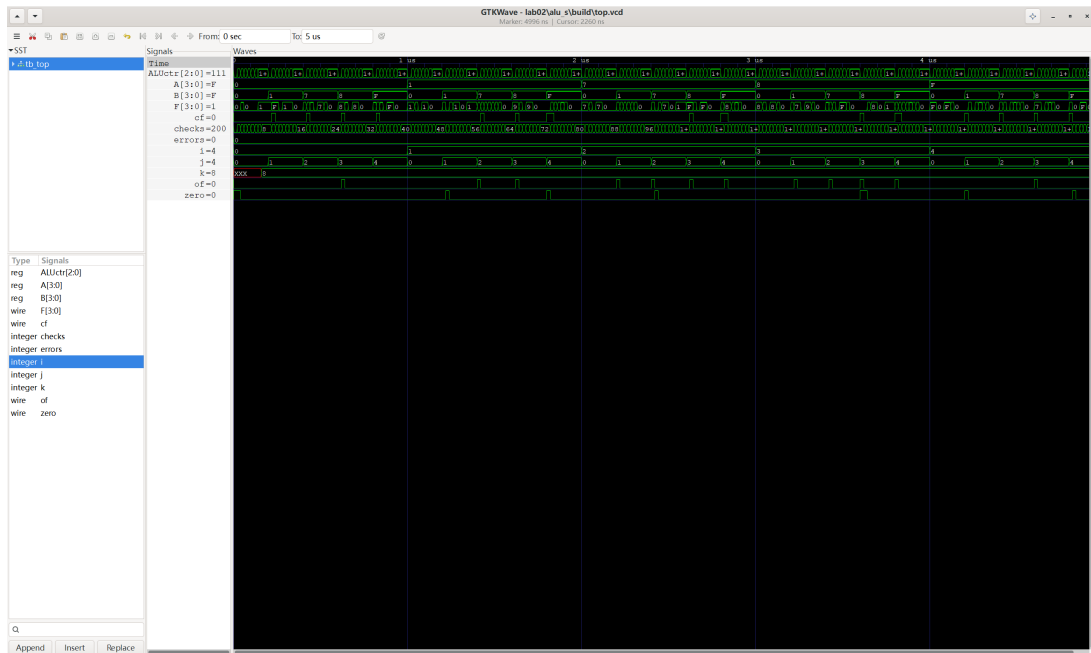


图 4: ALU 功能仿真波形

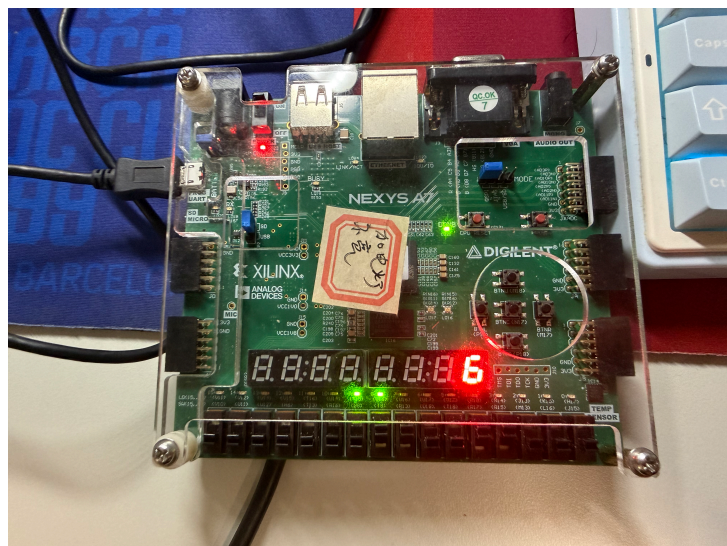


图 5: ALU 上板实测照片

8 问题与解决办法

- **标志信号残留**：casez 各分支若只对 F 赋值，前一次加/减遗留的 cf/zero/of 会串到随后的逻辑运算结果上。解决：在 always 块开头对 F 与三个标志统一清零，算术分支再按需赋值。
- **有符号小于需按补码解释**：直接写 $A < B$ 会把操作数当无符号数比较。解决：比较前用 \$signed() 把 4 位输入按二进制补码解释，再比较。
- **单位数码管如何显示符号/多位结果**：七段数码管只有一位且 bcd7seg 仅译 0-9，无法直接显示负号和十六进制结果。解决：top.v 中对负数求补码绝对值、显示数值大小，符号位由 LED[10:7] 的 $F[3]$ 判读，结果超出 0-9 时以 LED 二进制为准。

9 启示与建议

启示：

1. 熟练掌握 casez/always @(*) 描述多功能组合电路，以及“先清零后按分支赋值”避免状态残留的写法。
2. 理解了无符号进位 (cf) 与有符号溢出 (of) 是两套判据：前者看第 5 位，后者看操作数与结果的符号位，不能混用。
3. 自校验 testbench 用独立参考模型自动比对 200 组向量，比人工逐个看波形更可靠、可回归。

建议：单位数码管难以完整显示 4 位十六进制结果与符号。后续可改用 8 位数码管扫描显示、增加减法标志（是否负）提示位，让上板读数更直观。